

2. Editing Files

After learning how to create files and directories, let's move on to working with these files. There are several ways to edit a file in Linux, with some of the most common text editors being **Vi** and **Vim**. However, we will start with the **Nano** editor, which is less commonly used but easier to understand.

To create and edit a file using Nano, you can specify the file name directly as the first parameter when launching the editor. For example, to create and open a new file named **notes.txt**, you would use the following command:

```
dbcyph0n@htb[/htb]$ nano notes.txt
```

This command will open the **Nano** editor, allowing you to start editing the file **notes.txt** immediately. Nano's straightforward interface (also called "**pager**") makes it a great choice for quickly editing text files, especially when you're just getting started.

4. Nano Editor

```
GNU nano 2.9.3                               notes.txt

Here we can type everything we want and make our notes.

^G Get Help    ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos    M-U Undo
^X Exit        ^R Read File  ^\ Replace    ^U Uncut Text ^T To Spell   ^_ Go To Line M-E Redo
```

Below we see two lines with short descriptions. The **caret (^)** stands for our "**CTRL**" key. For example, if we press **[CTRL + W]**, a "**Search:**" line appears at the bottom of the editor, where we can enter the word or words we are looking for. If we now search for the word "**we**" and press **[ENTER]**, the cursor will move to the first word that matches.

```
GNU nano 2.9.3                               notes.txt

Here we can type everything we want and make our notes.

Search:  notes
^G Get Help  M-C Case Sens  M-B Backwards  M-J FullJstify ^W Beg of Par  ^Y First Line  ^P PrevHistory
^C Cancel    M-R Regexp     ^R Replace      ^T Go To Line  ^O End of Par  ^V Last Line   ^N NextHistory
```

To jump to the next match with the cursor, we press **[CTRL + W]** again and confirm with **[ENTER]** without any additional information.

```
GNU nano 2.9.3                               notes.txt

Here we can type everything we want and make our notes.

Search [we]:
^G Get Help  M-C Case Sens  M-B Backwards  M-J FullJstify ^W Beg of Par  ^Y First Line  ^P PrevHistory
^C Cancel    M-R Regexp     ^R Replace      ^T Go To Line  ^O End of Par  ^V Last Line   ^N NextHistory
```

Now we can save the file by pressing `[CTRL + O]` and confirm the file name with `[ENTER]`.

```
GNU nano 2.9.3                               notes.txt

Here we can type everything we want and make our notes.

File Name to Write: notes.txt
^G Get Help      M-C Case Sens  M-B Backwards  M-J FullJstify ^W Beg of Par  ^Y First Line  ^P PrevHstory
^C Cancel        M-R Regexp     ^R RepLace     ^T Go To Line  ^O End of Par  ^V Last Line   ^N NextHstory
```

After we have saved the file, we can leave the editor with `[CTRL + X]`.

4. Back on the Shell

To view the contents of the file, we can use the command `cat`.

```
dbcyph0n@htb[/htb]$ cat notes.txt

Here we can type everything we want and make our notes.
```

On Linux systems, there are several files that can be tremendously beneficial for penetration testers, due to misconfigured permissions or insufficient security settings by the administrators. One such important file is the `/etc/passwd` file. This file contains essential information about the users on the system, such as their usernames, user IDs (`UIDs`), group IDs (`GIDs`), and home directories.



Historically, the `/etc/passwd` file also stored password hashes, but now those hashes are typically stored in `/etc/shadow`, which has stricter permissions. However, if the permissions on `/etc/passwd` or other critical files are not set correctly, it may expose sensitive information or lead to privilege escalation opportunities.

As penetration testers, identifying files with improper rights or permissions can provide key insights into potential vulnerabilities that might be exploited, such as weak user accounts or misconfigured file access that should otherwise be restricted. Understanding these files is vital when assessing the security posture of a system.

2. VIM

`Vim` is an open-source editor for all kinds of ASCII text, just like Nano. It is an improved clone of the previous `Vi`. It is an extremely powerful editor that focuses on the essentials, namely editing text. For tasks that go beyond that, Vim provides an interface to external programs, such as `grep`, `awk`, `sed`, etc., which can handle their specific tasks much better than a corresponding function directly implemented in an editor usually can. This makes the editor small and compact, fast, powerful, flexible, and less error-prone.

Vim follows the Unix principle here: many small specialized programs that are well tested and proven, when combined and communicating with each other, resulting in a flexible and powerful system.

4. Vim

```
dbcyph0n@htb[/htb]$ vim
```

```

1 $
~
~      VIM - Vi IMproved
~
~          version 8.0.1453
~      by Bram Moolenaar et al.
~ Modified by pkg-vim-maintainers@lists.alioth.debian.org
~     Vim is open source and freely distributable
~
~           Sponsor Vim development!
~ type :help sponsor<Enter>   for information
~
~ type :q<Enter>               to exit
~ type :help<Enter> or <F1>    for on-line help
~ type :help version8<Enter>  for version info
~
~                                     0,0-1       All
```

In contrast to Nano, **Vim** is a modal editor that can distinguish between text and command input. Vim offers a total of six fundamental modes that make our work easier and make this editor so powerful:

Mode	Description
Normal	In normal mode, all inputs are considered as editor commands. So there is no insertion of the entered characters into the editor buffer, as is the case with most other editors. After starting the editor, we are usually in the normal mode.
Insert	With a few exceptions, all entered characters are inserted into the buffer.
Visual	The visual mode is used to mark a contiguous part of the text, which will be visually highlighted. By positioning the cursor, we change the selected area. The highlighted area can then be edited in various ways, such as deleting, copying, or replacing it.
Command	It allows us to enter single-line commands at the bottom of the editor. This can be used for sorting, replacing text sections, or deleting them, for example.
Replace	In replace mode, the newly entered text will overwrite existing text characters unless there are no more old characters at the current cursor position. Then the newly entered text will be added.
Ex	Emulates the behavior of the text editor Ex , one of the predecessors of Vim . Provides a mode where we can execute multiple commands sequentially without returning to Normal mode after each command.

When we have the Vim editor open, we can go into command mode by typing ":" and then typing "q" to close Vim.

```

1 $
~
~
~      VIM - Vi IMproved
~
~          version 8.0.1453
~      by Bram Moolenaar et al.
~ Modified by pkg-vim-maintainers@lists.alioth.debian.org
~     Vim is open source and freely distributable
~
~      Sponsor Vim development!
~ type :help sponsor<Enter>   for information
~
~ type :q<Enter>               to exit
~ type :help<Enter> or <F1>    for on-line help
~ type :help version8<Enter>   for version info
~
~ :q
```

Vim offers an excellent opportunity called **vimtutor** to practice and get familiar with the editor. It may seem very difficult and complicated at first, but it will only feel that way for a short time. The efficiency we gain from Vim once we get used to it is enormous. Entering the tutor mode in **vim** editor can be done using the **Command** mode **:Tutor** or by using the **vimtutor** command in the shell.

4. VimTutor

```
dbcyp0n@htb[/htb]$ vimtutor
```

```
=====
=  W e l c o m e   t o   t h e   V I M   T u t o r   -   V e r s i o n 1.7   =
=====

Vim is a very powerful editor that has many commands, too many to
explain in a tutor such as this.  This tutor is designed to describe
enough of the commands that you will be able to easily use Vim as
an all-purpose editor.

The approximate time required to complete the tutor is 25-30 minutes,
depending upon how much time is spent with experimentation.

ATTENTION:
The commands in the lessons will modify the text.  Make a copy of this
file to practice on (if you started "vimtutor" this is already a copy).

It is important to remember that this tutor is set up to teach by
use.  That means that you need to execute the commands to learn them
properly.  If you only read the text, you will forget the commands!

Now, make sure that your Caps-Lock key is NOT depressed and press
the  j  key enough times to move the cursor so that lesson 1.1
completely fills the screen.

~~~~~
```

5. ? Optional Exercise:

Play with the vimtutor. Get familiar with the editor and experiment with their features.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

100ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

← Previous

Next →

✓ Mark Complete & Next

 Cheat Sheet

5. Table of Contents

6. Introduction

Linux Structure



Linux Distributions



Introduction to Shell



6. The Shell

Prompt Description



Getting Help



 System Information



6. Workflow

 Navigation



 Working with Files and Directories



 Editing Files



 Find Files and Directories



 File Descriptors and Redirections



 Filter Contents

 Regular Expressions

 Permission Management

6. System Management

 User Management

 Package Management

 Service and Process Management

-  Task Scheduling
-  Network Services
-  Working with Web Services
-  Backup and Restore
-  File System Management
-  Containerization

6. Linux Networking

-  Network Configuration

Remote Desktop Protocols in Linux

6. Linux Hardening

Linux Security

-  Firewall Setup
-  System Logs and Monitoring

6. Linux Distributions vs Solaris

Solaris

6. Tips & Tricks

Shortcuts

5. My Workstation



OFFLINE

 Start Instance

∞ / 1 spawns left